

Inventory of Current State

- **Surfacer output:** `active_work = 0` and `next_automatic_action = NONE`. This means the system believes no tasks are currently running and no new task is automatically queued.
- **Visible items:** The surfacer shows multiple “surfaced” items, with the top item’s property `action_required = false`. In other words, there are items in view, but none is demanding immediate action under current rules.
- **Implication:** The system is in an apparently quiescent state: nothing is actively running or about to run, yet it has not declared “all work done.” Instead, it still sees one or more tasks (surfaced items), but none is flagged as requiring action.

Counterfactual States (Same Observations, Hidden Work)

We must consider plausible hidden obligations that would leave the surfacer output unchanged:

- **Unseen Dependencies:** Imagine an obligation (e.g. a pending code review or a background job) that the surfacer does not consider. For instance, a feature dependency might be needed before a surfaced item truly completes, but the surfacer doesn’t track it. The visible state remains `active_work=0, next_action=NONE` because the surfacer doesn’t know about the dependency, yet work is effectively unfinished.
- **Delayed or External Signals:** Perhaps an external trigger is needed to mark tasks as finished, but that trigger hasn’t arrived. The surfacer might treat them as done or ineligible for action (`action_required=false`), hiding the fact that a callback or message is missing. The surfacer still shows no next action because it has no information.
- **Phantom Authority or Input:** Maybe the system requires an explicit authorization or user input (not visible in the surfacer) before finalizing a task. Without that input, the task remains unresolved, but from the surfacer’s view, it simply sees an item that doesn’t auto-continue.

In these scenarios, the observations (`0, NONE, surfaced items present`) hold true, but **hidden obligations** remain. We need to detect such scenarios.

Distinguishing Quiescence vs. Hidden Obligations

To test if true quiescence is achieved, we design “probes” or minimal actions that could reveal hidden work:

- **Re-run or Refresh Observations:** Invoke the surfacer or related inspectors again. If new items appear or `action_required` toggles from false to true, the previous state was incomplete. A simple strategy is re-running the analysis that generated the surfacer output under a slightly different context or timestamp to catch any missed updates.

- **Inspect Unsurfaced Items:** Query the system for any tasks that did *not* surface. For example, check logs, message queues, or pending operations lists. If any expected tasks (like pending “Acknowledgements” or “Follow-ups”) exist but weren’t surfaced, then quiescence was false.
- **Forced Continuation:** Apply a “CONTINUE” action (as if prompted to keep working) and see what happens. If this triggers actual work (new tasks appear or current tasks change to needing action), then the previous state was prematurely considered quiescent.
- **Simulate External Events:** If some tasks depend on events (timers, inputs, or other processes), simulate these events and see if work resumes. For example, in a workflow engine, a “time-out” event might progress the state. If triggering such events causes changes, hidden dependencies existed.

Each experiment aims to preserve the known observations (surfacers still sees no active work, no auto-action, and the same visible items) but tests whether anything was concealed.

Surfacer Witness Reliability

We must examine what **the surfacer actually checks** versus what obligations it might miss:

- **What It Observes:** The surfacer sees the state of items (e.g., dispositions, action flags) and system status (active tasks, queued actions). It trusts that `active_work=0` means no jobs running, and `action_required=false` means no immediate manual action needed.
- **Possible Blind Spots:**
 - *Untracked Items:* If the system can hold tasks that never surface (e.g. orphaned tasks), those wouldn’t appear at all.
 - *State Synchronization:* If there’s a lag or race (an update hasn’t been picked up yet), the surfacer might see stale info.
 - *Observable vs. Unobservable Data:* The surfacer likely watches certain state fields. Any relevant field not monitored could hide obligation (for example, a “blocked” flag or an external signal channel).
- **Testing the Surfacers:** We should consider the surfacer as an “oracle” and test its completeness. For example:
 - Create a known unfinished scenario and confirm the surfacer catches it.
 - Conversely, simulate that hidden dependency example: start with a dependency not in surfacer’s scope, and see if surfacer wrongly stays at quiescence.

Synthesizing a Stopping Theorem

From these probes, we infer a condition for true quiescence:

Stopping Theorem (Tentative): *A state is truly quiescent exactly when it passes two tests: (1) The surfacer reports no active work and no immediate actions (`active_work=0`, `next_action=NONE`), and (2) re-running or lightly perturbing the system produces no new tasks or changes to surfaced items.*

In practice, this means we must ensure that **no unsurfaced, untriggered obligations exist that could affect outcomes**. If any experiment (refresh, continue, event simulation) causes changes, quiescence was not real.

Next bounded operations under quiescence:

- If all tests consistently show nothing new, we consider the system at rest. The next operations would likely be external (e.g., waiting for external stimulus or manual review), not automated tasks.
- If a test does reveal hidden work, then that work becomes the “actionable frontier,” and the system should shift back into active processing mode for that item.

Next Steps and Conclusion

Based on the analysis:

- **Actionable Insight:** Do not stop solely on `action_required=false`. Explicitly verify by running the appropriate probes (re-evaluate surfacer, simulate CONTINUE, inspect unsurfaced queues).
- **Potential Follow-up Actions:** If a hidden obligation is found, create a bounded task to resolve or surface it (e.g. fix observer logic, handle missing triggers). If none is found after all tests, the system can log quiescence and await any real input.

In summary, quiescence must be **earned** by confirming nothing remains unseen. A true quiescent state means no unresolved consequences exist beyond what’s visible—otherwise, our stopping condition is premature.
